

Infrastructure Migration & Architecture Blueprint: AWS to GCP

1. Core Objective

The primary goal is to migrate our existing infrastructure from AWS to Google Cloud Platform (GCP) by implementing a Containerized Modular Monolith. The architecture is designed to be Stateless and Serverless-ready, ensuring a seamless transition to Microservices in the future.

2. Infrastructure & Global Connectivity

To ensure low latency and edge-level security for a global user base, we will leverage Google's premium global network:

- **Global HTTP(S) Load Balancing:** Deployment of a single Anycast-based IP via Google's Premium Tier Network to route traffic to the nearest healthy instance.
- **Cloud Armor Integration:** Implementation of enterprise-grade security, including Web Application Firewall (WAF), Anti-DDoS protection, and Bot Management at the network edge.
- **Edge Computing:** Offloading lightweight processing and AI logic to the client-side (on-device) to minimize server round-trips and improve responsiveness.

3. Compute & Orchestration (Stateless Modular Monolith)

We are adopting a simplified yet powerful compute strategy to reduce operational overhead:

- **Google Cloud Run (Fully Managed):** Our primary compute engine. It provides a fully managed serverless environment for our containerized application, allowing for automatic horizontal scaling based on request volume.
- **Stateless Architecture:** All application components will be decoupled from the underlying infrastructure. No persistent data will be stored on individual instances; all state is offloaded to external managed services.

4. Data & State Management

Based on our technical requirements, we have selected MongoDB as our primary data store:

- **MongoDB Atlas (on GCP):** A fully managed NoSQL database service. We will utilize Global Sharding and multi-region clusters as our user base grows to ensure global data consistency and performance.
- **Cloud Memorystore (Redis):** A high-performance, in-memory service for distributed caching and session management to offload the primary database and reduce latency.
- **Google Cloud Storage:** All unstructured data (media files, assets) will be stored here, with only metadata and URLs managed within MongoDB.

Specific Requirements for MongoDB Implementation

To ensure our data layer is as robust as our compute layer, the following steps must be taken for the MongoDB setup:

1. Deployment via MongoDB Atlas on GCP:

- o The database must be hosted on **MongoDB Atlas**, specifically integrated within our GCP project regions to minimize cross-provider latency.
- o Enable **VPC Network Peering** or **Private Service Connect** between GCP and MongoDB Atlas to ensure all data traffic remains within the private network, never exposing the database to the public internet.

2. High Availability & Global Distribution:

- o **Replica Sets:** A minimum of a few-node replica set for automatic failover and high availability.
- o **Multi-Region Clusters:** (For future phases) Configured to support low-latency reads for our global users by placing data nodes in key geographic regions.

3. Security & Compliance:

- o **Encryption at Rest & in Transit:** Mandatory TLS/SSL for all connections.
- o **Role-Based Access Control (RBAC):** Strict IAM-integrated access policies to ensure the application has "least privilege" access to the collections.

- o **Network Isolation:** IP Whitelisting limited strictly to our Cloud Run service identities and office/admin IPs.

4. Scalability Strategy:

- o **Operational Monitoring:** Enable Atlas advanced telemetry to track query performance and index efficiency.
 - o **Sharding Readiness:** Data models should be designed with a clear "Shard Key" strategy in mind, allowing us to horizontally scale to billions of documents without downtime.
-

5. Scalability & Future-Proofing

To ensure the system remains robust and ready for a Microservices transition:

- **Event-Driven Architecture (Pub/Sub):** Implementation of Google Cloud Pub/Sub for asynchronous inter-module communication. This decouples time-consuming tasks (e.g., notifications, background processing) from the main request-response cycle.
- **Centralized Observability:** Utilizing Google Cloud Observability (Logging and Monitoring) for unified visibility across all modules.
- **CI/CD Pipeline:** Fully automated deployment workflows (e.g., GitHub Actions/Cloud Build) to push containerized updates to Cloud Run with zero downtime.

6. The AI "Brain" & Unified Intelligence

- **Generative AI (Gemini & Veo):** Direct in-app generation of text, high-quality images, and videos.
- **Smart Video Editor:** AI-powered editing to eliminate processing wait times and automate montage.
- **AI Search & Voice Assistant:** Natural language understanding and intelligent filtering with voice capabilities.
- **Unified Personalized AI Assistant & Analytics:** A cross-functional AI engine providing:
 - o **Personalized Guidance:** Tailored recommendations and guided shopping/interaction experiences.

- o **Strategic Analytics:** Real-time data visualization and decision-making support available for both users (personal insights) and business (market/operational insights).

7. Smart Feed: Personalized feed based on user interests and more (core logic we will develop together).

Smart Feed Architecture Strategy

To deliver a personalized and high-performance user experience, the Smart Feed will follow a decoupled, event-driven design:

- **Behavioral Tracking:** Interaction data (clicks, likes, dwell time) will be ingested via **GCP Pub/Sub** to ensure zero impact on main API latency.
- **Personalization Engine:** We will utilize **MongoDB Atlas Vector Search** to implement semantic content matching, allowing the feed to suggest content based on user interest patterns rather than just chronological order.
- **Performance Optimization:**
 - o **Ranking:** Hybrid scoring model (Recency + Engagement + User Affinity).
 - o **Caching:** Active user feeds will be partially pre-computed and stored in **Cloud Memorystore (Redis)** for sub-millisecond delivery.
 - o **Lazy Loading:** Cloud Run will handle the dynamic assembly of the feed, ensuring the system scales horizontally as concurrent users increase.

How it Works Together (The Workflow)

- o **Ingestion:** The system pulls a candidate pool of posts from 3 buckets:
 - o *Bucket A:* Paid Ads (Filtered by user location/demographics).
 - o *Bucket B:* Posts from followed accounts.
 - o *Bucket C:* AI-recommended posts (via Vector Search).
 - o **Weighted Merging:** The **Cloud Run** service merges these buckets.
 - o *Example Weighting:* Ads (40%), Following (40%), AI Recommendations (20%).
 - o **Final Ranking:** The feed is sorted to ensure the first few items the user sees are a mix of high-priority Ads and the most recent updates from their friends.
 - o **Delivery:** The final JSON is cached in **Redis** for 60 seconds to ensure that if the user refreshes, the experience is instantaneous.
-

8. Content Delivery & Egress Optimization

- **Google Media CDN:** Mandatory use of YouTube's global backbone for video delivery.
- **Adaptive Bitrate Streaming (ABS/HLS/DASH):** Mandatory integration of GCP Transcoder API for dynamic quality adjustment based on internet speed.
- **Static Assets Offloading:** Hosted on Cloud Storage (Multi-Regional) with **aggressive Edge Caching via Cloud CDN**.
- **Cloud Storage Signed URLs:** Secure, time-limited access to private media.

9. Messaging, Security & Observability

- **Cloud Pub/Sub:** High-throughput bus for inter-service communication and offloading of heavy background processing (AI training/Video rendering).
- **AI Targeting:** Advanced advertising tools from organic visibility to targeted paid ads.
- **Secret Manager:** Centralized management of API keys and credentials.
- **Infrastructure as Code (IaC):** Full automation via Terraform.
- **Cloud Operations Suite:** 360-degree observability (Logging, Monitoring, Trace, Error Reporting).

10. Deployment & CI/CD Pipeline

- **Artifact Registry:** Regionalized storage for Docker images with vulnerability scanning.
- **Zero-Downtime Deployment:** Mandatory Canary or Blue-Green strategies for 99.999% availability.
- **Binary Authorization:** Security gate ensuring only verified images are deployed.

11. Biometric Authentication & Face-ID Payment

Consumer Side: Local Authentication

- **Objective:** The user confirms the payment using their own smartphone.
- **Security:** Biometric data never leaves the device. Only the device-verified payment token is sent to the server.

Merchant Side: Face Recognition

- **Objective:** The business terminal or tablet recognizes the customer and processes the charge.
- **Technology:** Migration from Amazon Rekognition to Google Cloud Vision API.
- **Liveness Detection:** Implement Google ML Kit on the terminal to distinguish a real face from a photo or video (Anti-spoofing).

Data Security & Flow

- **Enrollment:** During user registration, generate a digital "fingerprint" (Face Embeddings) from the user's face using Google Cloud.
- **Privacy Rule:** Store only the vector models (hashes) in the database, never the actual photos of the user.

12. Business-to-business card payments

13. Minimize registration fields,

There is no need to manually include input fields for Country, State, City, or Username in this section. However, background Location Services must remain active at all times. The system should automatically resolve the user's location to power the hyper-local feed and business discovery without requiring manual user input.

14. We need to implement a simple multi-account (switch account) feature.

Scope:

- Users can add multiple accounts (multi-login)
- Users can switch between accounts without logging out
- Simple UI (profile icon → "Switch account")
- Fast switching between accounts

15. Images should have icons.

16. Add a new section within the business profile, allowing users to showcase products with their respective pricing.

=====If we are adding the showcase section we have to add the option for add the product, edit and update option.

Yes, like this